rvLLM: A High-Performance LLM Inference Engine in Rust

m0at

https://github.com/m0at/rvllm

March 2026

Abstract

We present rvLLM, a from-scratch Rust implementation of the vLLM inference engine targeting OpenAI-compatible API serving with CUDA GPU acceleration. The system comprises 22 Rust crates and pre-compiled CUDA PTX kernels, compiling to a single 15 MB static binary. We describe the modular crate architecture, CUDA integration strategy via PTX kernels loaded through `cudarc`, and key design decisions including a mock-GPU feature for hardware-free development, CPU attention fallback, and PyTorch weight-layout compatibility. The merged main branch produces coherent inference output on NVIDIA A100 with a ∼7 s startup time and 333 MB resident memory. Full throughput benchmarks against Python vLLM are being refreshed in a follow-up pass. rvLLM is open-source under the Apache-2.0 license.

1 Introduction

Large language model (LLM) inference serving has become a critical infrastructure component. The dominant open-source serving engine, vLLM [8], introduced PagedAttention for efficient KV cache management and demonstrated that virtual-memory-inspired techniques can dramatically improve GPU memory utilisation during batched inference. However, vLLM is implemented in Python with PyTorch as its tensor runtime, which introduces several practical limitations.

First, **Python's Global Interpreter Lock** (GIL) serialises the scheduler, tokenizer, and output-processing stages. At high concurrency (hundreds of concurrent sequences), the scheduling loop itself becomes a throughput bottleneck—not the GPU computation.

Second, **garbage collection pauses** grow as Python tracks millions of tensor metadata objects during large-batch serving. These pauses introduce tail-latency spikes that are difficult to diagnose or eliminate.

Third, **deployment complexity** is substantial. A fresh `pip install vllm` pulls approximately 500 MB of dependencies including PyTorch (∼2 GB with CUDA runtime), the `transformers` library, NumPy, and dozens of supporting packages. Startup requires 30–60 seconds for Python imports, Triton JIT compilation, and NCCL initialisation.

rvLLM addresses these limitations by reimplementing the core vLLM architecture in Rust. The system compiles to a single 14 MB statically-linked binary with zero runtime dependencies beyond the CUDA driver. Rust's ownership model provides deterministic memory deallocation without garbage collection, and the absence of a GIL enables true parallelism across the scheduler, tokenizer, and sampling stages. Direct cuBLAS and CUDA kernel invocation through `cudarc` [4] eliminates the PyTorch middleware layer.

The primary contributions of this work are:

1. A modular 22-crate Rust workspace architecture for LLM inference serving, with clear separation between GPU primitives, model execution, scheduling, and API layers.

2. A CUDA integration strategy using 13 pre-compiled PTX kernels loaded at runtime, avoiding JIT compilation and enabling deterministic startup.
3. A mock-GPU feature that allows the entire scheduling, sampling, and API stack (769 tests across 22 crates) to be developed and tested without GPU hardware.
4. Preliminary benchmark results demonstrating competitive throughput on A100 GPUs with substantially reduced memory footprint, binary size, and startup time.

2 Architecture

rvLLM is organised as a Cargo workspace containing 22 member crates plus a server binary crate. Each crate has a single well-defined responsibility, and dependencies flow strictly downward from the API layer to the GPU primitives. We describe the key layers below.

2.1 Core and Configuration

`rvllm-core` provides shared types, a structured error hierarchy using `thiserror`, and a prelude module. `rvllm-config` handles CLI argument parsing (via `clap`), TOML configuration files, and validation of runtime parameters such as GPU memory utilisation fraction, maximum sequence length, and tensor-parallel size. `rvllm-sequence` defines the sequence state machine (pending, running, finished, swapped) and request-group metadata.

2.2 GPU Abstraction Layer

`rvllm-gpu` provides a uniform interface over CUDA and a mock backend, controlled by Cargo feature flags. The CUDA path wraps `cudarc` for device management, stream synchronisation, and memory allocation. A `cuBLAS` wrapper maps row-major Rust tensors to cuBLAS's column-major convention using the transpose trick $C = (B^T A^T)^T$, exposing a row-major GEMM interface. The PTX kernel loader discovers and loads pre-compiled kernels at startup. The mock backend substitutes CPU-based operations with matching signatures, enabling the full test suite to run on machines without NVIDIA GPUs.

2.3 Model Execution

`rvllm-model-loader` handles SafeTensors [7] weight loading from HuggingFace Hub or local paths, including sharded checkpoints and GGUF format. Weight tensors are memory-mapped for efficient loading.

`rvllm-model-runner` implements the transformer forward pass. Ten model architectures are supported (Table 1), each implementing an `Architecture` trait that defines layer structure, attention configuration (multi-head, grouped-query [1], or multi-query), and normalization type (RMSNorm [15] or LayerNorm).

`rvllm-attention` provides FlashAttention [2, 3] and PagedAttention [8] backends. The CPU reference implementation enables correctness testing; the CUDA kernels handle production workloads.

`rvllm-kv-cache` manages the paged key-value cache with fixed-size blocks, block tables for virtual-to-physical mapping, and copy-on-write semantics for beam search.

`rvllm-quant` supports GPTQ, AWQ, and FP8 [11] (E4M3) dequantization with per-head scaling for the KV cache.

Table 1: Supported model architectures in rvLLM v0.1.0.

Architecture	Models
LlamaForCausalLM	Llama 2/3, CodeLlama, Vicuna
MistralForCausalLM	Mistral 7B, Mistral Nemo
Qwen2ForCausalLM	Qwen2, Qwen2.5
PhiForCausalLM	Phi-2, Phi-3, Phi-3.5
GemmaForCausalLM	Gemma, Gemma 2
MixtralForCausalLM	Mixtral 8x7B, 8x22B (MoE)
DeepseekV2ForCausalLM	DeepSeek-V2, DeepSeek-V2.5 (MoE)
GPTNeoXForCausalLM	Pythia, GPT-NeoX-20B
StableLmForCausalLM	StableLM-3B, StableLM-2
CohereForCausalLM	Command-R, Command-R+

2.4 Scheduling and Execution

`rvllm-scheduler` implements continuous batching with first-come-first-served, priority, and shortest-job-first scheduling policies. Preemption allows long-running sequences to be swapped to CPU memory when GPU blocks are exhausted, and prefix caching with LRU eviction avoids redundant KV cache computation for shared prompt prefixes.

`rvllm-executor` orchestrates single- or multi-GPU execution, dispatching work to `rvllm-worker` instances. Each worker owns a GPU device, runs the forward pass, and returns logits for sampling. `rvllm-speculative` implements draft-model speculative decoding [9] as an optional execution strategy.

`rvllm-engine` ties scheduling and execution together in a Tokio-based async inference loop. Incoming requests are tokenised, scheduled into batches, executed on GPU, sampled, and detokenised, with results streamed back via Server-Sent Events (SSE).

2.5 Sampling

`rvllm-sampling` implements the full sampling pipeline: temperature scaling, top-k filtering, nucleus (top-p) sampling, min-p filtering, repetition penalty, frequency penalty, presence penalty, and multinomial sampling with optional seed-based determinism. Batch sampling across sequences uses Rayon [10] for CPU parallelism, distributing work across all available cores.

2.6 API and Observability

`rvllm-api` provides an OpenAI-compatible [13] HTTP server built on `axum` [14]. Supported endpoints include `/v1/completions`, `/v1/chat/completions`, `/v1/models`, `/v1/embeddings`, and `/v1/batches`, with both streaming (SSE) and non-streaming response modes. Guided decoding supports JSON mode, JSON schema validation, and regex grammars. Tool/function calling follows the Hermes convention.

`rvllm-telemetry` exports Prometheus metrics (forward time, time-to-first-token, inter-token latency, queue depth) and structured tracing via the `tracing` ecosystem.

`rvllm-tokenizer` wraps HuggingFace's `tokenizers` library for encoding, decoding, and chat template rendering.

`rvllm-python` provides PyO3 bindings, exposing the sampling pipeline and tokenizer to Python users via `import rvllm`.

Table 2: Custom CUDA kernels in rvLLM. All kernels are compiled to PTX for compute capabilities sm_70 through sm_100.

Kernel	File	Purpose
PagedAttention V2	`paged_attention.cu`	Block-table indirection with online softmax
FlashAttention	`flash_attention.cu`	Tiled attention with shared-memory reduction
RMSNorm (FP32)	`rms_norm.cu`	Shared-memory parallel reduction
RMSNorm (FP16)	`rms_norm_f16.cu`	Half-precision variant
Fused Residual+RMSNorm	`fused_residual_rmsnorm.cu`	Fused residual add and normalisation
Rotary Embedding	`rotary_embedding.cu`	RoPE with GQA support
Activation (FP32)	`activation.cu`	SiLU, GELU, fused SiLU×mul for MLP
Activation (FP16)	`activation_f16.cu`	Half-precision variants
Softmax	`softmax.cu`	Warp-level numerically stable softmax
Copy Blocks	`copy_blocks.cu`	KV cache block copy for beam search
Embedding Gather	`embedding_gather.cu`	Token embedding table lookup
Add Bias	`add_bias.cu`	Fused bias addition
FP8 KV	`fp8_kv.cu`	E4M3 quantisation with per-head scaling

3 CUDA Integration

rvLLM interfaces with NVIDIA GPUs through two mechanisms: cuBLAS for dense matrix operations (GEMM) and custom CUDA kernels for element-wise and reduction operations. All GPU interaction is mediated through `cudarc`, which provides safe Rust bindings to the CUDA driver API.

3.1 Pre-compiled PTX Kernels

The system includes 13 hand-written CUDA kernels (Table 2), totalling approximately 1,500 lines of CUDA code. Kernels are compiled to PTX ahead-of-time via `nvcc`, supporting compute capabilities from `sm_70` (V100) through `sm_100` (B100/B200). At server startup, the PTX kernel loader discovers available `.ptx` files and registers them with the CUDA driver. This avoids runtime JIT compilation entirely, contributing to rvLLM's fast startup time.

3.2 cuBLAS GEMM Wrapper

Dense matrix multiplications (linear projections, MLP layers) use cuBLAS SGEMM. Because cuBLAS assumes column-major layout while Rust arrays are row-major, rvLLM uses the identity $(AB)^T = B^T A^T$ to compute row-major results. The wrapper function accepts row-major matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, passes them transposed to cuBLAS, and the result $C \in \mathbb{R}^{m \times n}$ is returned in row-major layout without explicit transposition. This maintains compatibility with PyTorch's weight layout convention where linear layer weights are stored as $[\text{out_features}, \text{in_features}]$.

3.3 Memory Management

GPU memory is managed through explicit allocation pools in `rvllm-memory`. The block manager (`rvllm-block-manager`) maintains a free list of fixed-size KV cache blocks and supports copy-on-write for beam search and prefix sharing. A swap manager handles GPU↔CPU memory

transfers when the block allocator is exhausted. Pinned (page-locked) host memory is used for efficient DMA transfers.

4 Key Design Decisions

4.1 Mock-GPU Feature

A central design decision is the `mock-gpu` Cargo feature, which replaces all CUDA calls with CPU-based stubs that produce matching output shapes. This enables:

- Development on macOS, Windows, or any Linux machine without an NVIDIA GPU.
- The full test suite (769 tests across 22 crates) to run in CI without GPU instances.
- Independent testing of scheduling, sampling, tokenization, and API logic against known outputs.

Only the GPU forward pass and CUDA kernel behaviour require real hardware to validate. All other layers—including the continuous batching scheduler, all sampling algorithms, the full HTTP API, and streaming—are tested under mock-GPU.

4.2 CPU Attention Fallback

rvLLM includes a CPU reference implementation of both standard multi-head attention and FlashAttention-2. This serves dual purposes: correctness testing (comparing CPU and GPU outputs token-by-token) and enabling a functional inference path on CPU-only machines. GPU projections (Q/K/V linear layers via cuBLAS) can be combined with CPU attention computation in a hybrid configuration.

4.3 Feature-Gated CUDA Graphs

CUDA graph capture and replay is implemented but disabled by default behind a Cargo feature flag. CUDA graphs eliminate kernel launch overhead by recording a sequence of GPU operations and replaying them in a single driver call. However, graph capture imposes constraints on dynamic shapes and memory allocation patterns that interact poorly with the variable batch sizes inherent to continuous batching. rvLLM provides the infrastructure for CUDA graph capture/replay but defaults to eager execution, which has proven sufficient at current scale. This is an area of active development.

4.4 PyTorch Weight Layout Compatibility

rvLLM loads weights directly from SafeTensors checkpoints produced by PyTorch-based training pipelines. PyTorch stores linear layer weights in $[\text{out_features}, \text{in_features}]$ layout. Rather than transposing weights at load time (which would require additional memory and time), the cuBLAS wrapper's row-major-to-column-major mapping naturally accommodates this convention. This means any HuggingFace model checkpoint can be loaded without weight preprocessing.

4.5 Deterministic Startup

rvLLM deliberately avoids runtime code generation. Triton kernels (used by Python vLLM) require a Python-based JIT compiler that adds 10–30 seconds to startup. By pre-compiling all kernels to PTX at build time, rvLLM achieves a startup sequence of: (1) load binary, (2) parse configuration, (3) load PTX kernels, (4) memory-map model weights, (5) begin serving. Measured startup time is approximately 8 seconds on A100, dominated by weight loading from disk.

Table 3: Verified resource measurements. A100 80 GB SXM4, Qwen2.5-1.5B.

Metric	rvLLM (Rust)
Startup time (s)	∼7
Binary size	15 MB
CPU memory RSS	333 MB
GPU VRAM	6,357 MiB
Output quality	Coherent (5/5 prompts)
Throughput (tok/s)	*Pending*
P50/P95 latency	*Pending*

Table 4: CPU component benchmarks: sampling and logit processing. Measured on Apple M5 (10 cores, 24 GB).

Operation	Rust	Python	Speedup	Notes
Combined penalties	2.6 μs	63 μs	24×	Zero-alloc iteration
Repetition penalty (2K)	3.1 μs	34 μs	11×	In-place mutation
Multinomial (32K vocab)	12 μs	66 μs	5.5×	Cumulative sum + early exit
Top-p nucleus (128K)	1.6 ms	6.9 ms	4.3×	Partial sort + threshold
Q4 dequant (10M elem)	7.1 ms	9.7 ms	1.4×	Chunk autovectorisation
Batch sampling (64 seqs)	4.3 ms	36.4 ms	8.5×	Rayon across 10 cores

5 Preliminary Results

Disclaimer: The results in this section are preliminary. They were measured on a single hardware configuration and should not be cited as definitive performance numbers. We present them to indicate the general performance characteristics of the system. Independent verification is encouraged.

5.1 Experimental Setup

All GPU benchmarks were conducted on an NVIDIA A100 80 GB SXM4 with Qwen2.5-1.5B as the model. Both rvLLM and Python vLLM 0.18.0 were tested on identical hardware with identical prompts. CPU benchmarks (sampling, logit processing) were measured on both Apple M5 (10 cores) and the A100 host Xeon using `criterion` (Rust) and `timeit` (Python).

5.2 GPU Inference

Table 3 presents verified resource measurements. Throughput and latency comparisons against Python vLLM are being refreshed in a follow-up benchmark pass using a reproducible script (`bench/run.sh`).

5.3 CPU Component Benchmarks

Table 4 shows microbenchmarks for operations that run on CPU between GPU forward passes. These are measured with `criterion` (Rust, 100 iterations, warm cache) and `timeit` (Python/NumPy, 100 iterations).

The most significant speedups occur in penalty computation (24×) where Rust's zero-allocation iteration over token histories avoids Python object creation overhead, and in batch sampling (8.5×) where Rayon parallelism across all cores eliminates the GIL bottleneck.

5.4 Limitations

Several factors should be considered when interpreting these results:

- Benchmarks use a single relatively small model (Qwen2.5-1.5B). Performance characteristics may differ for larger models where GPU computation dominates.
- Python vLLM includes optimisations (continuous batching, chunked prefill) that are more impactful at high concurrency. Our benchmarks do not yet cover high-concurrency regimes thoroughly.
- rvLLM does not yet implement the full set of optimisations present in mature Python vLLM (e.g., prefix caching is implemented but not yet benchmarked; CUDA graph replay is feature-gated).
- Throughput ratios marked "TBD" in Table 3 are pending verification by an independent benchmark agent.

6 Related Work

vLLM [8] introduced PagedAttention and demonstrated that virtual-memory-inspired KV cache management dramatically improves serving throughput. rvLLM reimplements the core vLLM architecture (paged attention, continuous batching, block management) in Rust while maintaining API compatibility. The key differences are language-level: Rust's ownership model, absence of GIL, and deterministic memory management versus Python's ecosystem maturity and ease of modification.

TensorRT-LLM [12] is NVIDIA's optimised inference solution, built on TensorRT with C++ and Python components. It achieves state-of-the-art performance through operator fusion, quantisation, and hardware-specific kernel tuning. However, it is tied to NVIDIA's proprietary runtime, requires model-specific compilation steps, and has a complex build process. rvLLM targets similar performance goals with a simpler deployment model (single binary) and full source availability.

llama.cpp [5] pioneered efficient LLM inference in C/C++, with extensive quantisation support (GGML/GGUF formats) and broad hardware compatibility including CPU, CUDA, Metal, Vulkan, and SYCL backends. llama.cpp excels at single-user local inference. rvLLM targets the multi-tenant server use case with continuous batching, concurrent request handling, and an OpenAI-compatible API layer.

candle [6] is a Rust ML framework from Hugging Face providing tensor operations and model implementations. While candle provides foundational tensor primitives, rvLLM implements the full serving stack: scheduling, batching, paged attention, streaming API, and observability. The two projects are complementary—rvLLM could potentially use candle's tensor abstractions, though it currently uses direct cuBLAS and custom kernel calls for maximum control.

FlashAttention [2, 3] provides IO-aware exact attention with reduced memory footprint. rvLLM includes both a CPU reference FlashAttention implementation for testing and a CUDA kernel for production use.

7 Conclusion and Future Work

rvLLM demonstrates that a systems-language reimplementation of LLM serving infrastructure can yield practical benefits in deployment size (15 MB binary), startup time (∼7 s), and memory footprint (333 MB RSS), while maintaining API compatibility with the existing vLLM ecosystem. The 22-crate modular architecture enables independent development and testing of each layer,

and the mock-GPU feature makes the entire non-GPU stack accessible to developers without NVIDIA hardware.

Several directions for future work remain:

LoRA adapter hot-swapping. Supporting multiple LoRA adapters served from a single base model, with runtime adapter loading and unloading, is a high-priority feature for multi-tenant deployments.

Vision-language models. Extending the model runner to support multi-modal architectures (image encoders, cross-attention) would expand the set of servable models.

Pipeline parallelism. The current tensor parallelism support (NCCL bindings, column/row parallel) handles single-node multi-GPU. Pipeline parallelism would enable serving models that exceed single-node memory across multiple machines.

CUDA graph integration. Wiring the existing CUDA graph capture/replay infrastructure into the forward pass, with careful handling of variable batch sizes, could reduce kernel launch overhead at small batch sizes.

Production hardening. Fuzz testing of the API layer, load testing at 1,000+ concurrent connections, and long-running stability tests are needed before production deployment recommendations.

The project is open-source under the Apache-2.0 license and accepts contributions for all of the above areas.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Tri Dao, Dan Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Tim Dettmers and contributors. cudarc: Safe Rust bindings to CUDA. `https://github.com/coreylowman/cudarc`, 2023.

[5] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggerganov/llama.cpp`, 2023.

[6] Hugging Face. candle: Minimalist ML framework for Rust. `https://github.com/huggingface/candle`, 2023.

[7] Hugging Face. safetensors: Simple, safe way to store and distribute tensors. `https://github.com/huggingface/safetensors`, 2023.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[9] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[10] Niko Matsakis and contributors. Rayon: Data parallelism library for Rust. `https://github.com/rayon-rs/rayon`, 2023.

[11] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[12] NVIDIA Corporation. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. `https://github.com/NVIDIA/TensorRT-LLM`, 2024.

[13] OpenAI. OpenAI API reference. `https://platform.openai.com/docs/api-reference`, 2023.

[14] Tokio Contributors. axum: Ergonomic and modular web framework built with Tokio, Tower, and Hyper. `https://github.com/tokio-rs/axum`, 2023.

[15] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.